Gates, Clocks and Lost Frames: Claude and Uppaal Probe the Robustness of TSN Gate Schedules

Author Names

Institution

Abstract. A modeller asked the AI assistant Claude, connected to Uppaal, to model and analyse the robustness of IEEE 802.1Qbv scheduled traffic in Time-Sensitive Networking (TSN), starting from a paper that formalises the constraints for window-based gate control lists [2]. The prompt fixed the modelling discipline: explicit transmission durations, stated traffic and fault assumptions, correlated clock uncertainty, packet identity and passive observers. Throughout, Claude *suggested and explained*: it designed a bounded network of timed automata, three gate schedules, the properties and the robustness experiments, and predicted their outcomes. Uppaal *verified and provided evidence*: verdicts and exact worst-case latencies for 29 configurations, concrete counterexamples, state-space statistics and, with Uppaal Smc, probability estimates with confidence intervals. The schedules are robust exactly up to the clock precision they were synthesised for; a schedule violating the paper's stream-isolation constraint fails even with perfect clocks; best-effort traffic requires length-aware gates or a guard band of one maximal frame. Uppaal also corrected Claude: one probability prediction was refuted, and three deviations of statistical results from exhaustive results and predictions exposed modelling errors in Claude's Smc variant. We summarise the session and reflect on which claims rest on verification and which on the assistant's reasoning.

1 Introduction

TSN extends Ethernet with deterministic, time-triggered communication. A *time-aware shaper* (IEEE 802.1Qbv [5]) opens and closes a gate in front of each egress queue according to a cyclic gate control list (GCL), executed against a clock synchronised by IEEE 802.1AS. Craciunas, Serna Oliver and Steiner [2] formalise the constraints a window-based GCL must satisfy so that every stream meets its end-to-end deadline and jitter bound, assuming a worst-case clock difference δ between any two nodes; the constraints are solved by an SMT solver. Whether a resulting schedule is *robust* – what happens if clocks are worse than assumed, applications release late, frames are lost or best-effort traffic interferes – is a question for an operational model. The session reported here started from the following prompt, given together with the paper:

Table 1. Roles in the session.

Claude *suggests and explains*	UPPAAL *verifies and provides evidence*	Modeller *decides*
reads the paper, designs the templates	syntax and type checks	poses the question
derives three GCLs from the constraints	verdicts: holds / violated	fixes the modelling
formalises properties as queries	exact worst-case latency (`sup`)	discipline
predicts outcomes analytically	shortest concrete counterexamples	asks for this
designs robustness experiments	SMC: probabilities with 95% CI	summary
explains verdicts and traces	states stored, CPU time, memory	

> *"I would like you to use UPPAAL to model analyse the robustness of scheduling traffic of the TSN protocol. You may consult the attached paper. I suggest to begin with small, bounded networks and explicit traffic generators, using templates such as:* `Source`*,* `Queue`*,* `Medium/Link`*,* `Scheduler`*,* `Receiver`*,* `FaultEnvironment`*, and* `Observer`*. The crucial modelling decisions are:*
>
> - *Model transmission duration explicitly. A channel synchronisation represents an instantaneous event, not the time needed to send a frame. Separate transmission start, medium occupancy and reception.*
> - *State traffic assumptions. Periods, minimum inter-arrival times, release jitter, priorities and maximum frame sizes determine whether deadline guarantees are possible.*
> - *State fault assumptions. Arbitrarily many losses or retransmissions usually defeat guaranteed delivery. Bounds on consecutive failures or a stochastic loss model answer different questions.*
> - *Treat clock uncertainty carefully. Bounded uncertainty in event times can give a useful over-approximation of drift, but independently choosing jitter at every event need not preserve the correlations of a physical oscillator.*
> - *Track packet identity. A deadline observer must match each release with its own delivery. A single clock reset on every release is unsound when messages overlap.*
> - *Keep observers passive. They must record deadline violations without blocking network actions or preventing time from progressing."*

Claude worked through a tool interface to the UPPAAL editor [1] (read and replace the model, check syntax, add and verify queries, read traces) and, for parameter sweeps, through UPPAAL's command-line verifier `verifyta` (version 5.1). Table 1 shows how the work was divided. Throughout this summary, verdicts, latencies, traces, probabilities and statistics are UPPAAL's reports; the model, the schedules, the formalisation of properties, the predictions and the explanations are Claude's suggestions, and we say so where the distinction matters.

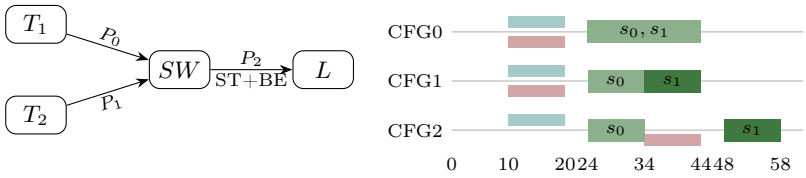


Fig. 1. Left: the bounded network. Right: the three GCLs within one cycle of local time. Thin bars: windows on P_0 (upper) and P_1 (lower); thick bars: ST windows on P_2 with their assigned frames. Outside the ST windows the best-effort (BE) gate of P_2 is open.

2 The Case: Window-Based Gate Schedules

In the paper's model [2] a stream s_i is a periodic frame from a talker to a listener along a fixed route; on each directed link the frame has transmission time L. Every egress port has one queue for scheduled traffic (ST) whose gate is open during windows $w_k = [w_k.open, w_k.close]$, and a boolean $w_k.\phi(f_i)$ assigns each frame to exactly one window. The constraints used here are: (i) windows on a link are ordered and sized by their frames, $w_k.close = w_k.open + \sum_i w_k.\phi(f_i) \cdot L$; (ii) consecutive hops of a stream respect the synchronisation error, $w_k^{[v_j,v_{j+1}]}.close + \delta \leq w_l^{[v_{j+1},v_{j+2}]}.open$; (iii) two streams arriving on different links and leaving on the same port are assigned to the same window or *isolated* in time, i.e. the second may enter the switch only after the first has left it [3]; and (iv) end-to-end latency and jitter are bounded via the window boundaries, with δ added once.

The bounded instance (Claude). Claude chose the smallest network that exercises both sharing and isolation: talkers T_1, T_2, a switch SW and a listener L, with egress ports $P_0 = [T_1, SW]$, $P_1 = [T_2, SW]$ and $P_2 = [SW, L]$ (Fig. 1). Streams s_0 (from T_1) and s_1 (from T_2) have period 100, equal to the cycle, frame time $L = 10$ on every link and a per-hop propagation and store-and-forward delay $PD = 2$. Claude derived three GCLs by hand from the constraints for $\delta = 2$; for instance $20 + PD + \delta = 24$ is the earliest admissible window on P_2:

- **CFG0** (shared window): P_0, P_1: $[10, 20]$; P_2: one window $[24, 44]$ for both frames. All constraints hold; deadline $E2E = 48$.
- **CFG1** (no isolation): as CFG0, but P_2 has $w_0 = [24, 34]$ for s_0 and $w_1 = [34, 44]$ for s_1. Both frames are in the switch at the same time, so constraint (iii) is *violated*. $E2E = 38$ for s_0 and 48 for s_1.
- **CFG2** (time isolation): P_1: $[34, 44]$, with s_1 released at 24; P_2: $w_0 = [24, 34]$ for s_0, $w_1 = [48, 58]$ for s_1. All constraints hold; $E2E = 38$ for both streams.

3 The Model (Claude)

Claude built a network of timed automata from ten templates, instantiated once per node clock, talker, port or link: Uppaal creates one process per value of a

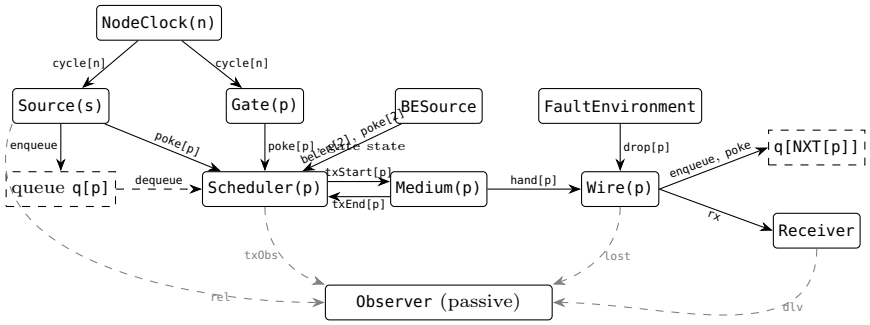


Fig. 2. Communication between the processes of one port p (three ports, two sources, three node clocks). Solid: synchronisations and shared data; dashed grey: broadcasts received by the observer, which never sends. The BE source feeds only port P_2; `Wire(p)` delivers to the next port's queue (P_0, P_1) or to the listener (P_2).

template parameter, giving 22 processes. Fig. 2 shows how they communicate, Table 2 maps them to the modelling decisions of the prompt, and Figs. 3–5 show the automata exactly as in the model, in UPPAAL's colour scheme: `select`, `guard`, `synchronisation`, `update`, `invariant`; initial locations are drawn double, committed locations carry a C, urgent ones a U. All robustness dimensions are constants at the top of the declarations – `CFG`, `DELTA` (actual clock precision Δ), `DRIFT`, `RJ` (release jitter), `LOSS`, `MAXCONS`, `BE`, `GB` (guard band), `LA` (length-aware selection) and `EVJ` – so that a sweep rewrites one line. Value passing uses shared variables written by the sender of a synchronisation and read by its receivers (`txFrame`, `wireFrame`, `rxFrame`); channel arrays such as `rel[s][seq]` carry the identity of a frame instance in the channel index.

3.1 Time and Traffic: `NodeClock` and `Source`

`NodeClock(n)` (Fig. 3, top) gives node n its own notion of time. In the committed location `Init` it chooses the node's offset `off[n]` from $\{0, \ldots, \Delta\}$; in `Start` it waits exactly that long and then starts the first cycle by resetting the local clock `lt[n]` and broadcasting `cycle[n]`. In `Run` the clock counts local time since the start of the current cycle. The wrap at the end of a cycle is the only place where the offset may change: the select `d` chooses a change of at most `DRIFT`, the guard keeps the offset in $[0, \Delta]$, and the equality `lt[n] == HP + d` lets the cycle last $HP + d$, so that all later events of the node are shifted by `d`. The invariant `lt[n] <= HP + dmax(n)` forces the wrap. Because every gate and source of a node reads the same clock, an offset shifts all of the node's events together – the correlation of a physical oscillator that the prompt asks for. With `DRIFT = 0` the offset is constant in a run, the classical bounded-offset model; with `DRIFT = 1` it may change by one unit per cycle, in either direction, chosen nondeterministically. Pairwise differences never exceed Δ, the precision assumed by the paper.

Table 2. Templates and the modelling decisions of the prompt they implement.

Template	Behaviour	Decision addressed
`NodeClock(n)`	local cycle clock `lt[n]`; offset `off[n]` in $[0, \Delta]$, may change by at most `DRIFT` per cycle	correlated clock uncertainty
`Source(s)`	releases instance (`s`,`seq`) once per cycle in $[REL, REL{+}RJ]$	traffic assumptions; identity
`Gate(p)`	executes the GCL on its node's clock: ST and BE gate states, window index, closing times	802.1Qbv time-aware shaper
`Scheduler(p)`	strict priority ST over BE among open gates, FIFO, optional length-aware check	transmission selection
`Medium(p)`	*occupies* the link for the frame length	explicit duration
`Wire(p)`	propagation PD, then *reception* into the next queue, or loss	start, occupancy, reception
`Receiver`	listener; announces the delivery of (`s`,`seq`)	identity
`BESource`	sporadic BE frames on P_2, lengths 4..8, minimum inter-arrival 20	interference assumptions
`FaultEnvironment`	drops frames on wires, at most `MAXCONS` consecutive instances per stream	bounded fault assumption
`Observer`	per-instance deadline clocks, window and overrun checks; broadcast only, input-enabled	passive observation
queue (data)	FIFO arrays `q[p][QCAP]` with `enqueue`/`dequeue`; one BE slot `beLen[p]` with tail drop	– (an automaton would add interleavings, no behaviour)

`Source(s)` (Fig. 3, middle) is the talker application of stream s. On each `cycle[n]` of its node it enters `Wait`, whose invariant together with the guard of the outgoing edge releases the frame at some local time in $[REL, REL{+}RJ]$, i.e. with release jitter `RJ`. The release edge broadcasts `rel[s][k]` – the instance's identity is the channel index – and puts the frame identifier `2*s+k` into the queue of the first port. From the committed location `Rel` it then pokes that port's scheduler and toggles the sequence bit `k`. A one-bit sequence number suffices because the observer checks that no more than two instances of a stream are ever in flight.

3.2 The Time-Aware Shaper: `Gate` and `Scheduler`

`Gate(p)` (Fig. 3, bottom) executes the GCL of port p on the clock of the port's node. Each ST window w contributes three events, enumerated by `ph`: the BE gate closes (`GB` before the window), the ST gate opens, and the ST gate closes while the BE gate reopens. The function `evT(p,ph)` returns the local time of event `ph`; the invariant `lt[PNODE[p]] <= evT(p,ph) + EVJ` together with the

guard `lt[PNODE[p]] >= evT(p,ph)` fires it exactly on time (`EVJ = 0`) or, in the naive clock model, up to `EVJ` late, independently of all other events. `gateEvent` updates the gate state that the scheduler reads: `stOpen`, `beOpen`, the index `curWin` of the open window and the local closing times `stClose` and `beClose`. Every event pokes the scheduler. On `cycle[n]` the gate restarts the list via the committed location `CycleStart`.

`Scheduler(p)` (Fig. 4, top) implements 802.1Qbv transmission selection: among the queues whose gate is open, ST has strict priority over BE, and each queue is served in FIFO order. A poke moves it from `Idle` to the *urgent* location `Decide`, where no time may pass; there, mutually exclusive guards decide. If the head ST frame may start – the ST gate is open, the queue is non-empty and, for length-aware selection, the frame fits before the gate closes, `lt[PNODE[p]] <= stLatest(p)` with `stLatest(p) = stClose[p] - LEN[s]` – the scheduler dequeues it and sends `txStart[p]` to the medium; the committed location `NotifyST` then reports the start to the observer on `txObs[p][s]`. Two further edges start a BE frame when the BE gate is open and no ST frame is eligible; four edges with complementary guards return to `Idle` when nothing is eligible (e.g. a frame that would not fit is *held*). In `Tx` the scheduler waits for `txEnd[p]` and decides again. With `LA = false` the bounds `stLatest` and `beLatest` become unbounded: a frame may start whenever its gate is open and overrun its close. Since eligibility can only be lost, never gained, while time passes within a window, deciding only at pokes – gate events, arrivals, ends of transmissions – is exact, and the urgency makes the scheduler work-conserving: it never idles while a frame could be sent.

3.3 Transmission: `Medium`, `Wire` and `Receiver`

The prompt's first decision is to separate transmission start, medium occupancy and reception; these are three different templates.

`Medium(p)` (Fig. 4, middle left) models the occupancy of the link. `txStart[p]?` copies frame and length and resets `x`; the invariant `x <= len` and guard `x >= len` keep it in `Busy` for exactly the transmission time. It then signals `txEnd[p]!` to the scheduler and, from the committed location `Handover`, passes the frame to the wire with `hand[p]!`. Both channels are binary: if the medium were not idle at `txStart`, or the wire not free at `hand`, the system would deadlock, so `A[] not deadlock` also checks these two modelling assumptions.

`Wire(p)` (Fig. 4, middle right) models the propagation and store-and-forward delay `PD`: the frame is *received* only after `Prop`, when the whole frame has crossed the link. Then one of four edges applies: a ST frame on P_0 or P_1 is enqueued at the switch and pokes the scheduler of P_2; a ST frame on P_2 is handed to the listener with `rx!`; a BE frame is consumed; or the fault environment drops the frame (`drop[p]?`), and the committed location `Lost` announces the loss of that instance on `lost[s][seq]`.

Receiver (Fig. 4, bottom left) is the listener. It accepts the frame, announces the delivery of the instance on `dlv[s][seq]` and resets the stream's count of consecutive losses.

3.4 Environment: `BESource` and `FaultEnvironment`

BESource (Fig. 5, left) is a sporadic best-effort talker at the switch. Its single edge may fire at any time at least `BE_MIA` after the previous arrival – there is no invariant, so arrivals may also stop – and chooses a frame length in $[BE_MINL, BE_MAXL]$. The BE queue holds one frame; arrivals at a full queue are dropped, which only affects BE traffic.

FaultEnvironment (Fig. 5, middle) is the adversary for losses. It may drop the frame of any wire at the moment of reception, but only while the stream has fewer than `MAXCONS` consecutive lost instances; a delivery resets the count. This is the prompt's "bound on consecutive failures". Since dropping is optional and delivery is always possible, the fault environment never blocks the network.

3.5 Passive Observation: `Observer`

Observer (Fig. 5, right) matches each release with its own delivery. It keeps one clock `age[s][seq]` and one flag `pending[s][seq]` per frame instance: `rel[i][j]?` resets the clock and sets the flag – and sets `obsOverflow` if that instance was still pending, which would mean that the observer's two slots per stream do not suffice. `lost[i][j]?` clears the flag, so that a lost frame is not counted as late. On `dlv[i][j]?` the committed location `CheckDelivery` compares the instance's age with its deadline. For frames that never arrive, an internal edge records the miss as soon as the age exceeds the deadline:

```
select i : sid_t, j : int[0,1]
guard  pending[i][j] && !dlMiss[i] && age[i][j] > E2E[CFG][i]
update dlMiss[i] = true
```

On `txObs[p][s]?` the committed location `CheckTx` checks the transmission against the GCL: `wrongWin` if the frame's assigned window `ASSIGN[CFG][p][s]` is not the open one, `overrun` if it cannot finish before the window closes. The observer is passive by construction: it only *receives* on broadcast channels, which never block a sender; it is input-enabled, since every receiving edge has a trivially true guard; and its committed locations have complementary guards, so it leaves them immediately and never stops time.

3.6 Properties (Claude)

Five safety properties and, per stream and slot, a worst-case latency query:

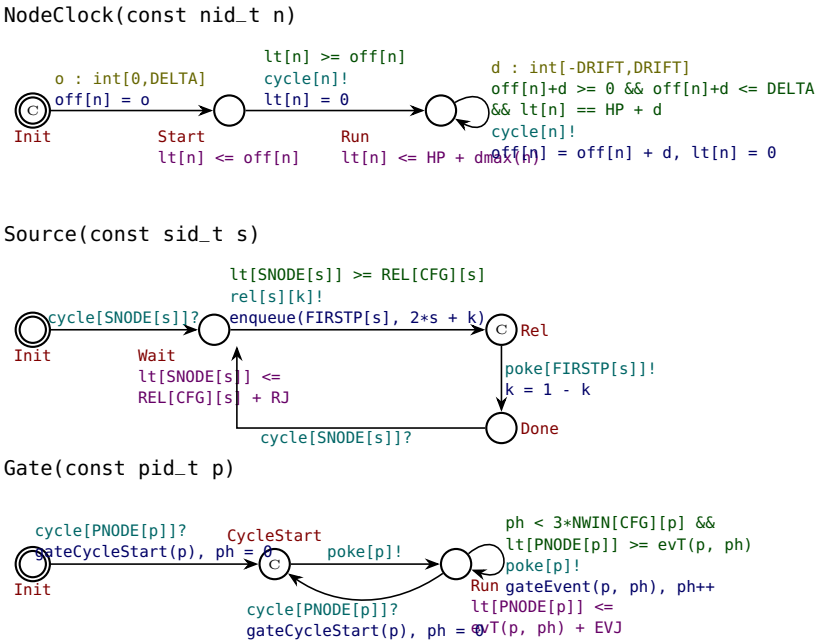


Fig. 3. The automata `NodeClock`, `Source` and `Gate`. The self-loop of `NodeClock` is the cycle wrap, the only point where a node's offset can change; the self-loop of `Gate` executes the next GCL event.

```
A[] not deadlock
A[] !dlMiss[0] && !dlMiss[1]
A[] forall(p : pid_t) forall(s : sid_t) !wrongWin[p][s]
A[] forall(p : pid_t) forall(s : sid_t) !overrun[p][s]
A[] !qOverflow && !obsOverflow
sup{pending[0][0]} : age[0][0]
```

The `sup` query asks Uppaal for the supremum of an instance's age while it is pending, i.e. the exact worst-case end-to-end latency.

4 Exhaustive Verification (Uppaal)

Predictions (Claude). Before verifying, Claude derived the expected bounds for CFG0. The second frame is received at $44 + PD$ in switch time and was released at 0 in talker time, so the worst-case latency should be $46 + \Delta$, and the schedule should break at $\Delta = 3$; with release jitter 11 it should break because the slack before the window opens is 10. Table 3 lists Uppaal's reports for all exhaustive runs. Every exhaustive prediction was confirmed.

Reading the table (Claude). The schedules are robust exactly up to the precision they were synthesised for, and the worst-case latency grows linearly with

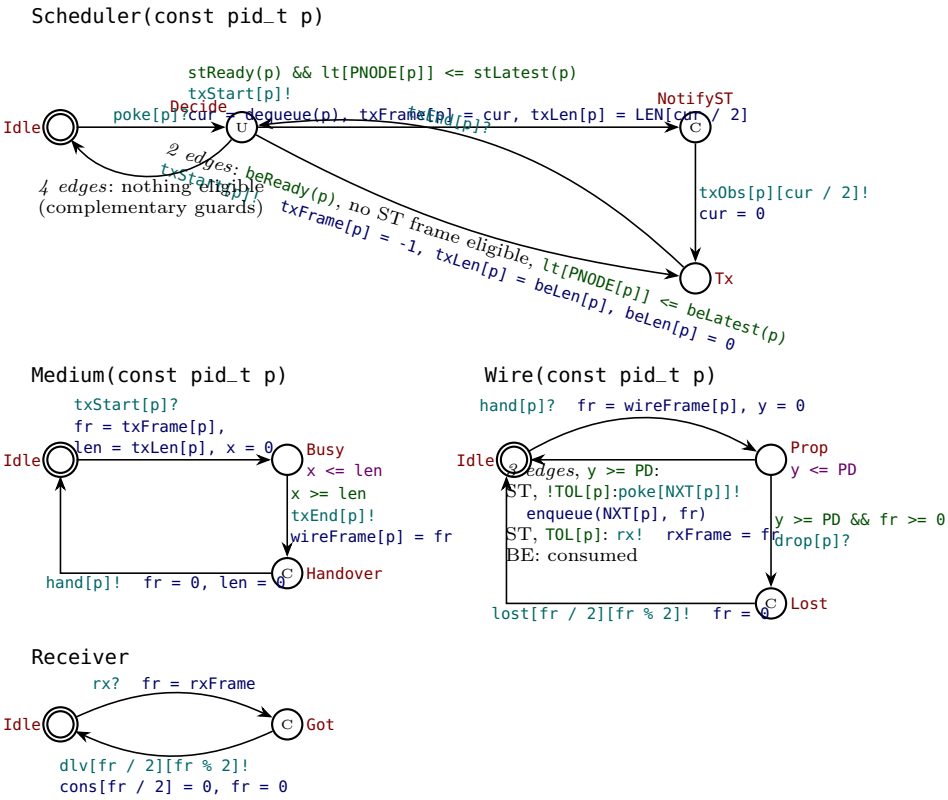
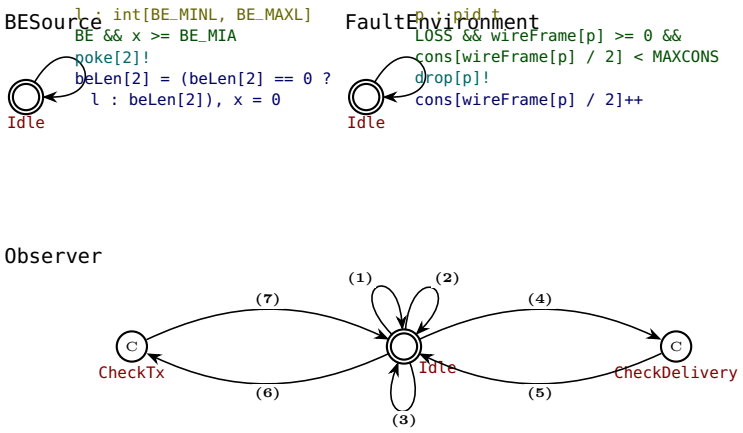


Fig. 4. The automata `Scheduler`, `Medium`, `Wire` and `Receiver`. Edges with complementary guards are summarised (in italics); all guards appear verbatim in the model file. Transmission start (`txStart`), medium occupancy (`Busy`) and reception (end of `Prop`) are separate steps.



(1) `i : sid_t, j : int[0,1]; pending[i][j] && !dlMiss[i] && age[i][j] > E2E[CFG][i]; dlMiss[i] = true` (internal: frame never arrives) (3)

(5) `i : sid_t, j : int[0,1]; dlv[i][j]?; cs = i, ck = j`

(7) `pp : pid_t, i : sid_t; txObs[pp][i]?; cp = pp, cs = i`

Fig. 5. The environment (`BESource`, `FaultEnvironment`) and the passive `Observer`, whose edges are listed below the drawing. The observer only receives, on broadcast channels, and its committed locations have complementary guards: it can neither block the network nor stop time.

Δ; bounded drift within Δ changes nothing. CFG1 fails *with perfect clocks*: the isolation constraint is necessary, not merely sufficient. With at most three consecutive losses the isolated schedules keep every delivered frame on time and in its window – delivery itself is, of course, not guaranteed. BE traffic is harmless under length-aware selection; without it the worst-case latency is $48 + 8 - GB$, so a guard band of one maximal BE frame is necessary and sufficient. The naive clock model reports violations even for $\Delta = 0$: independent jitter at the open and the close event shrinks a window that a physical clock would only shift. As for cost, most configurations are verified in seconds; sporadic BE arrivals dominate the state space, and the largest runs stored up to $7 \cdot 10^6$ states and used up to 1.3 GB of memory and 17 minutes of CPU time.

Counterexamples (UPPAAL). For every violated property Claude requested UPPAAL's shortest diagnostic trace (option `-t1`). UPPAAL returned concrete traces with explicit delays, which Claude condensed into the event lists of Table 4; times are global, offsets are those chosen in the trace. Fig. 6 draws three of them as timelines over the three egress ports and the listener.

Explanations (Claude). (a) Both frames reach the switch at 22; in the interleaving where s_1 arrives first it is at the head of the FIFO and is sent in s_0's window – the concurrent-arrival scenario that the isolation constraint excludes. (b) The talkers' clocks run 3 ahead of the switch; both frames arrive at 25, after the shared window opened at 24; the second cannot finish before 44 and waits for the next cycle. (c) A release at 10.5 misses the 10-unit window $[10, 20]$ by half a unit. (d) A late gate opening at 10.5 has the same effect with perfect clocks: the spurious behaviour of the independent-jitter model. (e) A BE frame starting at 21 occupies the link until 25, beyond the BE gate's close at 24; s_1 then starts at 35 and overruns the window. Each explanation is read off UPPAAL's trace.

5 Statistical Model Checking (UPPAAL SMC)

The exhaustive engine answers *whether* a violation is possible; the prompt's remark on stochastic fault models asks *how likely* it is. In UPPAAL SMC [4] nondeterministic choices are resolved uniformly and delays in bounded intervals are uniform, so a clock offset is uniform on $\{0, \ldots, \Delta\}$ and a release uniform on $[REL, REL{+}RJ]$. SMC requires broadcast channels, so Claude prepared an SMC variant in which the binary channels are broadcast – sound here because the exhaustive deadlock check showed that their receivers are always ready – with losses disabled, and with exponential BE inter-arrivals of rate 1/10 after the minimal distance. As deadline property Claude used the observer's condition directly, `exists(i : sid_t) exists(j : int[0,1]) (pending[i][j] && age[i][j] > E2E[CFG][i])`. Table 5 lists the estimates; each took between 1 and 40 seconds of CPU time (`verifyta`, seed 1).

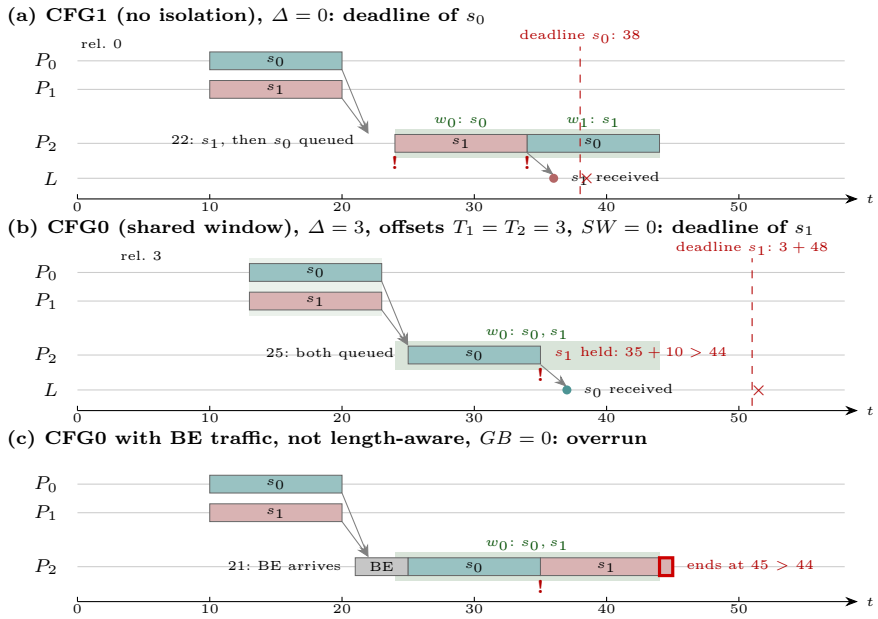


Fig. 6. Three of UPPAAL's counterexamples as timelines (global time; bars are transmissions on the egress ports, grey arrows the propagation delay $PD = 2$, light green the ST windows of the GCL; **!** marks an observer flag, × the recorded deadline miss). (a) Shortest trace for the deadline of s_0 (329 states, < 1 ms): the two frames reach the switch simultaneously, s_1 is queued first and is sent in s_0's window, s_0 is pushed into w_1, and both transmissions are flagged `wrongWin`; s_0 misses its deadline of 38. (b) Trace (b) of Table 4: the talkers run 3 ahead, both frames arrive at 25, after the window opened, and s_1 is held until the next cycle. (c) Trace (e): a BE frame started at 21 occupies the link beyond the BE gate's close at 24 and pushes s_1 past the window's close. Bars are drawn to the end of the transmission, which the frame length determines, even where a trace stops earlier.

A refuted prediction. For CFG0 with $\Delta = 3$, Claude predicted that a miss requires both talkers to be 3 ahead of the switch, i.e. probability 1/64. UPPAAL SMC estimated $[0.027, 0.038]$, which excludes 1/64. Claude then asked the exhaustive engine, for each of the 64 offset combinations, whether a miss is reachable. UPPAAL answered yes for exactly two: $(3, 3, 0)$, the predicted case, and $(0, 0, 3)$, where the *switch* is 3 ahead. Claude explained the second: both frames still fit into the window, but it opens 3 late in global time, so the latency becomes $46 + 3 = 49 > 48$ – the end-to-end constraint accounts for δ only once. The corrected value $2/64 \approx 0.031$ lies in UPPAAL's interval; the release-jitter and arrival-order predictions were confirmed.

Reading the table (Claude). Beyond the verdicts, the estimates rank the failure modes. With constant offsets the risk at $\Delta = 3$ is a fixed 3% per deployment;

with drifting offsets it accumulates to over 70% in 100 cycles. A schedule without isolation fails in half the cycles. Without length-aware selection a guard band of 4 still leaves a 4% chance per cycle, and $GB = 7$ leaves a rare event (3 in 2000 runs of ten cycles) that random simulation hardly sees but the exhaustive engine found in half a second: SMC quantifies, it does not replace verification.

When the engines disagreed. The first SMC runs contradicted an exhaustive result or a prediction three times, and each time the cause was Claude's SMC variant, not the system. (1) All BE runs reported probability 0, although UPPAAL had produced BE counterexamples. A query for any BE transmission also returned 0: Claude's BE source combined an exponential rate with a clock guard (`x >= BE_MIA`) on the same edge, which in this form never fired. The source was split into a wait of exactly `BE_MIA` followed by an exponential delay, which has the same timed behaviours. (2) The release-jitter run reported 0 for one cycle. The observer's timeout edge had the same defect, so misses were only recorded at delivery, in the next cycle; using the observer condition as the query removed the dependence. (3) Drift made no difference over 100 cycles. Claude attributed this to the equality guard `lt[n] == HP + d`: a uniformly drawn delay hits a point with probability 0, so only the choice at the invariant's bound was ever taken and all offsets drifted to Δ. The variant now selects the next drift at the start of a cycle and waits for exactly that time; the estimates rose from 0.03 to 0.14 (ten cycles) and to 0.72 (100 cycles). The SMC numbers in Table 5 are from the corrected variant. The diagnoses of (1) and (3) are Claude's explanations; what UPPAAL established is that the original variant never fired the BE source and that the corrected one does.

6 Reflections

What rests on verification, and what does not. Every verdict, latency, trace, probability and statistic in this summary is UPPAAL's. Claude's contributions are of three kinds: the model and its abstractions, the claim that the queries express the intended properties, and the explanations. The first two are open to inspection; the explanations were checked against UPPAAL's traces wherever a property failed. Where Claude only argued – that the hand-derived GCLs satisfy the paper's constraints, that a queue capacity of 3 and two observer slots per stream suffice – UPPAAL confirmed the consequences (no overflow in any run) but not the argument.

Predict, then verify. Stating the expected numbers *before* running UPPAAL turned verification into a test of Claude's understanding. The exhaustive predictions matched; one statistical prediction did not, and the mismatch revealed a property of the schedule that Claude had overlooked: a late switch is as harmful as late talkers. The assistant's reasoning is cheap and fallible; the verifier's evidence decides.

Two engines check each other. The SMC variant was Claude's translation of the exhaustive model, and three of its defects were found only because SMC disagreed with verdicts and counterexamples of the symbolic engine. A statistical

result that is never cross-checked against an exhaustive one – or against a prediction – can be confidently wrong: an interval of $[0, 0.0018]$ for a configuration that fails in 70% of the runs.

The prompt's modelling decisions mattered. Each one is visible in the results. Separating start, occupancy and reception made the length-aware check and BE overruns expressible. Correlated per-node offsets versus independent per-event jitter is the difference between "robust up to $\Delta = 2$" and "broken at $\Delta = 0$". Instance-indexed deadline clocks and the overflow check make the latencies of 133 and 148 – frames carried into the next cycle while the next instance has already been released – meaningful. Bounded consecutive losses gave a property that holds; unbounded losses would only have shown that delivery cannot be guaranteed. And a stochastic view answered a different question than the bounded one, as the prompt anticipated.

Limitations (Claude's assessment). The window observer compares window indices, not cycles. The paper's isolation constraint compares window boundaries of different nodes without δ; in this instance the slack absorbs it, and Claude did not search for a schedule that exposes it. The independent-jitter variant is expensive for the symbolic engine and was used only for counterexample search. The SMC variant does not model losses; a probabilistic loss model with branching probabilities is the natural next step, together with multiple periods (Section 4 of [2]) and larger topologies.

Practicalities. The tool interface was used to build the model, check its syntax and run the first verification; `verifyta` of the same version produced the sweeps, traces, statistics and SMC estimates. Before replacing the model open in the editor, Claude saved it to a backup file. The modeller posed the question and the modelling discipline and asked for this summary; Claude suggested and explained; UPPAAL verified and provided the evidence. This summary was written by Claude from the session record.

References

1. Behrmann, G., David, A., Larsen, K.G.: A tutorial on UPPAAL. In: SFM-RT 2004. LNCS, vol. 3185, pp. 200–236. Springer (2004)
2. Craciunas, S.S., Serna Oliver, R., Steiner, W.: Formal scheduling constraints for time-sensitive networks. arXiv:1712.02246 (2017)
3. Craciunas, S.S., Serna Oliver, R., Chmelík, M., Steiner, W.: Scheduling real-time communication in IEEE 802.1Qbv time sensitive networks. In: RTNS 2016, pp. 183–192. ACM (2016)
4. David, A., Larsen, K.G., Legay, A., Mikučionis, M., Poulsen, D.B.: UPPAAL SMC tutorial. Int. J. Softw. Tools Technol. Transf. 17(4), 397–415 (2015)
5. IEEE: 802.1Qbv – Enhancements for scheduled traffic. IEEE Std 802.1Qbv-2015 (2016)

Table 3. UPPAAL's reports (`verifyta` 5.1, breadth-first, default extrapolation). Verdicts of the deadline, window and overrun properties; deadlock freedom and absence of overflow held in every run. WCRT: result of the `sup` queries for s_0/s_1. States: largest number stored for any of the ten queries; CPU: all ten queries.

Experiment	Setting	deadline	window	overrun	WCRT	states	CPU [s]
precision	CFG0, $\Delta = 0$	holds	holds	holds	46/46	1 752	0.1
	CFG0, $\Delta = 1$	holds	holds	holds	47/47	6 444	0.5
	CFG0, $\Delta = 2$	holds	holds	holds	48/48	15 625	1.1
	CFG0, $\Delta = 3$	**viol.**	holds*	holds	133/133	32 741	2.4
	CFG0, $\Delta = 4$	**viol.**	holds*	holds	133/133	63 488	9.2
release jitter	CFG0, $RJ = 5$	holds	holds	holds	48/48	48 501	8.1
	CFG0, $RJ = 10$	holds	holds	holds	48/48	79 965	20.0
	CFG0, $RJ = 11$	**viol.**	holds*	holds	148/148	1 041 281	251.5
drift	CFG0, $\Delta = 2$, $DRIFT = 1$	holds	holds	holds	48/48	30 670	4.4
	CFG0, $\Delta = 2$, $DRIFT = 1$, $RJ = 10$	holds	holds	holds	48/48	119 969	55.6
	CFG0, $\Delta = 3$, $DRIFT = 1$	**viol.**	holds*	holds	139/139	371 824	65.6
no isolation	CFG1, $\Delta = 0$	**viol.**	**viol.**	holds	46/46	2 570	0.4
	CFG1, $\Delta = 2$	**viol.**	**viol.**	holds	48/48	22 163	2.5
	CFG1, $\Delta = 3$	**viol.**	**viol.**	holds	133/133	43 508	4.3
time isolation	CFG2, $\Delta = 0$	holds	holds	holds	36/36	714	<0.1
	CFG2, $\Delta = 2$	holds	holds	holds	38/38	9 946	0.8
	CFG2, $\Delta = 3$	**viol.**	**viol.**	holds	60/112	22 057	1.6
loss	CFG0, $MAXCONS = 1$	holds	holds	holds	48/48	89 651	14.4
	CFG0, $MAXCONS = 3$	holds	holds	holds	48/48	455 047	86.1
	CFG1, $MAXCONS = 1$	**viol.**	**viol.**	holds	48/48	231 053	29.1
	CFG2, $MAXCONS = 1$	holds	holds	holds	38/38	56 868	6.6
	CFG2, $MAXCONS = 3$	holds	holds	holds	38/38	301 592	58.7
BE interference	CFG0, length-aware, $GB = 0$	holds	holds	holds	48/48	1 151 058	271.7
	CFG0, not l.-a., $GB = 0$	**viol.**	holds	**viol.**	56/56	6 045 054	1009.3
	CFG0, not l.-a., $GB = 4$	**viol.**	holds	**viol.**	52/52	6 981 851	539.6
	CFG0, not l.-a., $GB = 7$	**viol.**	holds	**viol.**	49/49	3 355 829	271.7
	CFG0, not l.-a., $GB = 8$	holds	holds	holds	48/48	1 103 053	181.1
naive clocks	CFG0, $\Delta = 0$, $EVJ = 1, 2$	**viol.**	–	–	–	1 364†	<0.1†

*The window observer compares window indices, not cycles; a frame pushed into the same window of the next cycle is caught by the deadline property only. †Depth-first search for the deadline property alone; a full breadth-first exploration did not finish within 15 minutes.

Table 4. Counterexamples returned by UPPAAL, condensed by Claude. States and CPU time are UPPAAL's statistics for finding the trace.

Property, setting	Events of the trace (global time)
(a) window; CFG1, $\Delta = 0$ 290 states, 16 ms	0: s_0, s_1 released; 10: both talkers start; 22: s_1 enqueued at SW, then s_0; 24: P_2 starts s_1 in w_0 – *frame outside its assigned window*
(b) deadline; CFG0, $\Delta = 3$ 6 640 states, 78 ms	offsets T_1=3, T_2=3, SW=0; 3: releases; 13: both talkers start; 25: both enqueued; 25: P_2 starts s_0; 35: s_1 held, does not fit before 44; 37: s_0 received; 51.5: *deadline miss of* s_1
(c) deadline; CFG0, $RJ = 11$ 29 676 states, 0.7 s	10.5: s_0 released, held (window $[10, 20]$ too short); 11: s_1 released, held; 58.75: *deadline miss*
(d) deadline; CFG0, $\Delta = 0$, $EVJ = 1$ 1 364 states (DFS), 32 ms	0: releases; 10.5: both gates open late, frames held (window $[10.5, 20]$ too short); 48.5: *deadline miss*
(e) overrun; CFG0, BE, not l.-a., $GB = 0$ 19 772 states, 0.5 s	10: both talkers start; 21: BE frame of length 4 arrives and starts; 22: both ST frames enqueued; 25: s_0 starts; 35: s_1 starts – *ends at 45, after the window closes at 44*

Table 5. UPPAAL SMC estimates of `Pr[<=T; N](<> deadline miss)` with the 95% confidence intervals reported by UPPAAL ($T = 100$ is one cycle). Predictions are Claude's; "–" means none was made.

Setting	T	runs	misses	95% CI	prediction
CFG0, $\Delta = 2$	1000	4000	0	$[0, 0.0009]$	0
CFG0, $\Delta = 3$	1000	4000	127	$[0.027, 0.038]$	$1/64 \approx 0.016$ **refuted**; corrected $2/64 \approx 0.031$
CFG0, $\Delta = 3$	10000	1000	25	$[0.016, 0.037]$	0.031 (offsets constant)
CFG0, $\Delta = 3$, $DRIFT = 1$	1000	4000	569	$[0.132, 0.153]$	–
CFG0, $\Delta = 3$, $DRIFT = 1$	10000	1000	725	$[0.696, 0.752]$	grows with T
CFG0, $\Delta = 2$, $DRIFT = 1$	10000	1000	0	$[0, 0.0037]$	0
CFG0, $RJ = 11$	100	4000	667	$[0.155, 0.179]$	$1 - (10/11)^2 \approx 0.174$
CFG0, $RJ = 11$	1000	4000	3386	$[0.835, 0.858]$	$1 - (10/11)^{20} \approx 0.851$
CFG1, $\Delta = 0$	100	4000	1980	$[0.479, 0.511]$	0.5 (arrival order)
CFG1, $\Delta = 2$	100	4000	1985	$[0.481, 0.512]$	–
BE, length-aware, $GB = 0$	1000	2000	0	$[0, 0.0018]$	0
BE, not l.-a., $GB = 0$	100	4000	994	$[0.235, 0.262]$	–
	1000	2000	1443	$[0.701, 0.741]$	–
BE, not l.-a., $GB = 4$	100	4000	165	$[0.035, 0.048]$	–
	1000	2000	313	$[0.141, 0.173]$	–
BE, not l.-a., $GB = 7$	100	4000	0	$[0, 0.0009]$	–
	1000	2000	3	$[0.0003, 0.0044]$	–
BE, not l.-a., $GB = 8$	1000	2000	0	$[0, 0.0018]$	0